



Go from Data to Strategy ▶

ONLINE MASTER OF SCIENCE IN
BUSINESS ANALYTICSCarnegie Mellon University
Tepper School of Business

Go from Data to Strategy: Tepper School of Business

Loss Functions in PyTorch Models

by **Adrian Tam** on [April 8, 2023](#) in **Deep Learning with PyTorch** 💬 0[Share](#)[Share](#)[Post](#)

The loss metric is very important for neural networks. As all machine learning models are one optimization problem or another, the loss is the objective function to minimize. In neural networks, the optimization is done with gradient descent and backpropagation. But what are loss functions, and how are they affecting your neural networks?

In this chapter, you will learn what loss functions are and delve into some commonly used loss functions and how you can apply them to your neural networks. After reading this chapter, you will learn:

- What are loss functions, and their role in training neural network models
- Common loss functions for regression and classification problems
- How to use loss functions in your PyTorch model

Kick-start your project with my book [Deep Learning with PyTorch](#). It provides **self-study tutorials** with **working code**.

Let's get started!



Loss Functions in PyTorch Models.
Photo by Hans Vivek. Some rights reserved.

Overview

This post is divided into four sections; they are:

- What Are Loss Functions?
- Loss Functions for Regression

- Loss Functions for Classification
- Custom Loss Function in PyTorch

What Are Loss Functions?

In neural networks, loss functions help optimize the performance of the model. They are usually used to measure some penalty that the model incurs on its predictions, such as the deviation of the prediction away from the ground truth label. Loss functions are usually differentiable across their domain (but it is allowed that the gradient is undefined only for very specific points, such as $x = 0$, which is basically ignored in practice). In the training loop, they are differentiated with respect to parameters, and these gradients are used for your backpropagation and gradient descent steps to optimize your model on the training set.

Loss functions are also slightly different from metrics. While loss functions can tell you the performance of our model, they might not be of direct interest or easily explainable by humans. This is where metrics come in. Metrics such as accuracy are much more useful for humans to understand the performance of a neural network even though they might not be good choices for loss functions since they might not be differentiable.

In the following, let's explore some common loss functions, for regression problems and for classification problems.

Want to Get Started With Deep Learning with PyTorch?

Take my free email crash course now (with sample code).

Click to sign-up and also get a free PDF Ebook version of the course.

Download Your FREE Mini-Course

Loss Functions for Regression

In regression problems, the model is to predict a value in a continuous range. Too good to be true that your model can predict the exact value all the time, but it is good enough if the value is close enough. Therefore, you need a loss function to measure how close it is. The farther away from the exact value, the more the loss is your prediction.

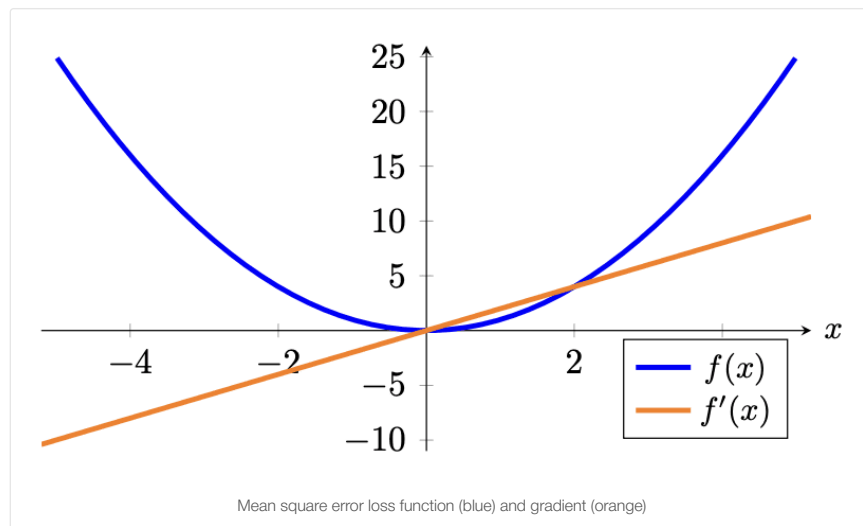
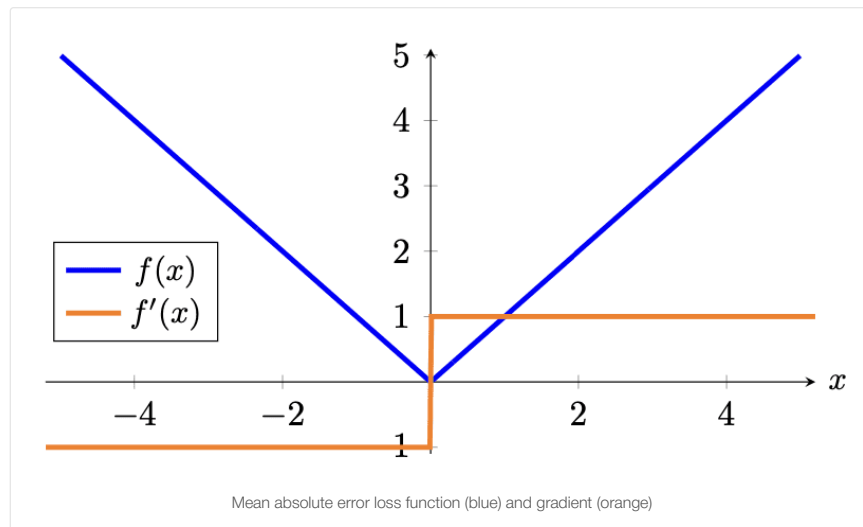
One simple function is just to measure the difference between the prediction and the target value. You do not care the value is greater than or less than the target value in finding the difference. Hence, in mathematics, we find $\frac{1}{m} \sum_{i=1}^m |\hat{y}_i - y_i|$ with m the number of training examples whereas y_i and $\hat{y}_i$ are the target and predicted values, respectively, averaged over all training examples. This is the mean absolute error (MAE).

The MAE is never negative and would be zero only if the prediction matched the ground truth perfectly. It is an intuitive loss function and might also be used as one of your metrics, specifically for regression problems, since you want to minimize the error in your predictions.

However, absolute value is not differentiable at 0. It is not really a problem because you rarely hitting that value. But sometimes people would prefer to use mean square error (MSE) instead. MSE equals to $\frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$, which is similar to MAE but use square function in place of absolute value.

It also measures the deviation of the predicted value from the target value. However, the MSE squares this difference (always non-negative since squares of real numbers are always non-negative), which gives it slightly different properties. One property is that the mean squared error favors a large number of small errors over a small number of large errors, which leads to models with fewer outliers or at least outliers that are less severe than models trained with a MAE. This is because a large error would have a significantly larger impact on the error and, consequently, the gradient of the error when compared to a small error.

Let's look at what the mean absolute error and mean square error loss function looks like graphically:



Similar to activation functions, you might also be interested in what the gradient of the loss function looks like since you are using the gradient later to do backpropagation to train your model's parameters. You should see that in MSE, larger errors would lead to a larger magnitude for the gradient and a larger loss. Hence, for example, two training examples that deviate from their ground truths by 1 unit would lead to a loss of 2, while a single training example that deviates from its ground truth by 2 units would lead to a loss of 4, hence having a larger impact. This is not the case in MAE.

In PyTorch, you can create MAE and MSE as loss functions using `nn.L1Loss()` and `nn.MSELoss()` respectively. It is named as L1 because the computation of MAE is also called the L1-norm in mathematics. Below is an example of computing the MAE and MSE between two vectors:

```
1 import torch
2 import torch.nn as nn
3
4 mae = nn.L1Loss()
5 mse = nn.MSELoss()
6
7 predict = torch.tensor([0., 3.])
8 target = torch.tensor([1., 0.])
9
10 print("MAE: %.3f" % mae(predict, target))
11 print("MSE: %.3f" % mse(predict, target))
```

You should get

```
1 MAE: 2.000
2 MSE: 5.000
```

MAE is 2.0 because $\frac{1}{2}[|0 - 1| + |3 - 0|] = \frac{1}{2}(1 + 3) = 2$ whereas MSE is 5.0 because $\frac{1}{2}[(0 - 1)^2 + (3 - 0)^2] = \frac{1}{2}(1 + 9) = 5$. Notice that in MSE, the second example with a predicted value of 3 and actual value of 0 contributes 90% of the error under the mean squared error vs. 75% under the mean absolute error.

Sometimes, you may see people use root mean squared error (RMSE) as a metric. This will take the square root of MSE. From the perspective of a loss function, MSE and RMSE are equivalent. But from the perspective of the value, the RMSE is in the same unit as the predicted values. If your prediction is money in dollars, both MAE and RMSE give you how much your prediction is away from the true value in dollars in average. But MSE is in unit of squared dollars, which its physical meaning is not intuitive.

Loss Functions for Classification

For classification problems, there is a small, discrete set of numbers that the output could take. Furthermore, the number used to label-encode the classes is arbitrary and with no semantic meaning (e.g., using the labels 0 for cat, 1 for dog, and 2 for horse does not represent that a dog is half cat and half horse). Therefore, it should not have an impact on the performance of the model.

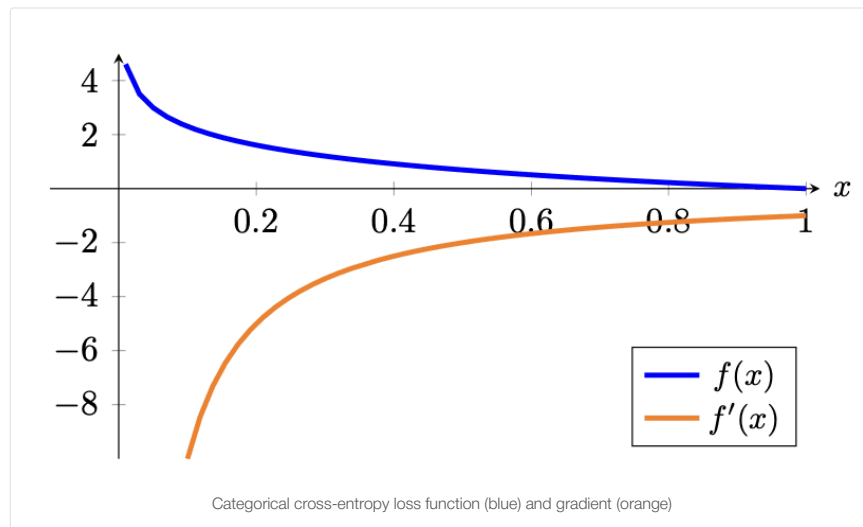
In a classification problem, the model's output is usually a vector of probability for each category. Often, this vector is usually expected to be "logits," i.e., real numbers to be transformed to probability using the softmax function, or the output of a softmax activation function.

The cross-entropy between two probability distributions is a measure of the difference between the two probability distributions. Precisely, it is $-\sum_i P(X = x_i) \log Q(X = x_i)$ for probability P and Q . In machine learning, we usually have the probability P provided by the training data and Q predicted by the model, which P is 1 for the correct class and 0 for every other class. The predicted probability Q , however, is usually a floating point valued between 0 and 1. Hence when used for classification problems in machine learning, this formula can be simplified into:

$$\text{categorical cross-entropy} = -\log p_{\text{target}}$$

where p_{target} is the model-predicted probability of the ground truth class for that particular sample.

Cross-entropy metrics have a negative sign because $\log(x)$ tends to negative infinity as x tends to zero. We want a higher loss when the probability approaches 0 and a lower loss when the probability approaches 1. Graphically,



Notice that the loss is exactly 0 if the probability of the ground truth class is 1 as desired. Also, as the probability of the ground truth class tends to 0, the loss tends to positive infinity as well, hence substantially penalizing bad predictions. You might recognize this loss function for logistic regression, which is similar except the logistic regression loss is specific to the case of binary classes.

Looking at the gradient, you can see that the gradient is generally negative, which is also expected since, to decrease this loss, you would want the probability on the ground truth class to be as high as possible. Recall that gradient descent goes in the opposite direction of the gradient.

In PyTorch, the cross-entropy function is provided by `nn.CrossEntropyLoss()`. It takes the predicted logits and the target as parameter and compute the categorical cross-entropy. Remind that inside the `CrossEntropyLoss()` function, softmax will be applied to the logits hence you should not use softmax activation function at the output layer. Example of using the cross entropy loss function from PyTorch is as follows:

```
1 import torch
2 import torch.nn as nn
3
4 ce = nn.CrossEntropyLoss()
5
6 logits = torch.tensor([[[-1.90, -0.29, -2.30], [-0.29, -1.90, -2.30]])
7 target = torch.tensor([[0., 1., 0.], [1., 0., 0.]])
8 print("Cross entropy: %.3f" % ce(logits, target))
```

It prints:

```
1 Cross entropy: 0.288
```

Note the first argument to the cross entropy loss function is logit, not probabilities. Hence each row does not sum to 1. The second argument, however, is a tensor containing rows of probabilities. If you convert the `logits` tensor above into probability using softmax function, it would be:

```
1 probs = torch.tensor([[0.15, 0.75, 0.1], [0.75, 0.15, 0.1]])
```

which each row sums to 1.0. This tensor also reveals why the cross entropy above calculated to be 0.288, which is $-\log(0.75)$.

The other way of calculating the cross entropy in PyTorch is not to use one-hot encoding in the target but to use the integer indices label:

```
1 import torch
2 import torch.nn as nn
3
4 ce = nn.CrossEntropyLoss()
5
6 logits = torch.tensor([[ -1.90, -0.29, -2.30], [ -0.29, -1.90, -2.30]])
7 indices = torch.tensor([1, 0])
8 print("Cross entropy: %.3f" % ce(logits, indices))
```

This gives you the same cross entropy of 0.288. Note that,

```
1 import torch
2
3 target = torch.tensor([[0., 1., 0.], [1., 0., 0.]])
4 indices = torch.argmax(target, dim=1)
5 print(indices)
```

gives you:

```
1 tensor([1, 0])
```

This is how PyTorch interprets your target tensor. It is also called “sparse cross entropy” function in other libraries, to make a distinction that it does not expect a one-hot vector.

Note in PyTorch, you can use `nn.LogSoftmax()` as an activation function. It is to apply softmax on the output of a layer and then take the logarithm on each element. If this is your output layer, you should use `nn.NLLLoss()` (negative log likelihood) as the loss function. Mathematically these two are the same as cross entropy loss. You can confirm this by checking the code below produced the same output:

```
1 import torch
2 import torch.nn as nn
3
4 ce = nn.NLLLoss()
5
6 # softmax to apply on dimension 1, i.e. per row
7 logsoftmax = nn.LogSoftmax(dim=1)
8
9 logits = torch.tensor([[ -1.90, -0.29, -2.30], [ -0.29, -1.90, -2.30]])
10 pred = logsoftmax(logits)
11 indices = torch.tensor([1, 0])
12 print("Cross entropy: %.3f" % ce(pred, indices))
```

In case of a classification problem with only two classes, it becomes binary classification. It is special because the model is now a logistic regression model in which there can be only one output instead of a vector of two values. You can still implement binary classification as multiclass classification and use the same cross entropy function. But if you output x as the probability (between 0 and 1) for the “positive class”, it is known that the probability for the “negative class” must be $1 - x$.

In PyTorch, you have `nn.BCELoss()` for binary cross entropy. It is specialized for binary case. For example:

```
1 import torch
2 import torch.nn as nn
3
4 bce = nn.BCELoss()
5
6 pred = torch.tensor([0.75, 0.25])
7 target = torch.tensor([1., 0.])
8 print("Binary cross entropy: %.3f" % bce(pred, target))
```

This gives you:

```
1 Binary cross entropy: 0.288
```

It is because:

$$\frac{1}{2} [-\log(0.75) + (-\log(1 - 0.25))] = -\log(0.75) = 0.288$$

Note that in PyTorch, the target label 1 is taken as the “positive class” and label 0 is the “negative class”. There should not be other values in the target tensor.

Custom Loss Function in PyTorch

Notice in above, the loss metric is calculated using an object from `torch.nn` module. The loss metric computed is a PyTorch tensor, so you can differentiate it and start the backpropagation. Therefore, nothing forbids you from creating your own loss function as long as you can compute a tensor based on the model’s output.

PyTorch does not give you all the possible loss metrics. For example, mean absolute percentage error is not included. It is like MAE, defined as:

$$\text{MAPE} = \frac{1}{m} \sum_{i=1}^m \left| \frac{\hat{y}_i - y_i}{y_i} \right|$$

Sometimes you may prefer to use MAPE. Recall the example on regression on the California housing dataset, the prediction is on the house price. It may make more sense to consider prediction accurate based on percentage difference rather than dollar difference. You can define your MAPE function, just remember to use PyTorch functions to compute, and return a PyTorch tensor.

See the full example below:

```

1 import copy
2
3 import matplotlib.pyplot as plt
4 import numpy as np
5 import pandas as pd
6 import torch
7 import torch.nn as nn
8 import torch.optim as optim
9 import tqdm
10 from sklearn.model_selection import train_test_split
11 from sklearn.datasets import fetch_california_housing
12 from sklearn.preprocessing import StandardScaler
13
14 # Read data
15 data = fetch_california_housing()
16 X, y = data.data, data.target
17
18 # train-test split for model evaluation
19 X_train_raw, X_test_raw, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)
20
21 # Standardizing data
22 scaler = StandardScaler()
23 scaler.fit(X_train_raw)
24 X_train = scaler.transform(X_train_raw)
25 X_test = scaler.transform(X_test_raw)
26
27 # Convert to 2D PyTorch tensors
28 X_train = torch.tensor(X_train, dtype=torch.float32)
29 y_train = torch.tensor(y_train, dtype=torch.float32).reshape(-1, 1)
30 X_test = torch.tensor(X_test, dtype=torch.float32)
31 y_test = torch.tensor(y_test, dtype=torch.float32).reshape(-1, 1)
32
33 # Define the model
34 model = nn.Sequential(
35     nn.Linear(8, 24),
36     nn.ReLU(),
37     nn.Linear(24, 12),
38     nn.ReLU(),
39     nn.Linear(12, 6),
40     nn.ReLU(),
41     nn.Linear(6, 1)
42 )
43
44 # loss function and optimizer
45 def loss_fn(output, target):
46     # MAPE loss
47     return torch.mean(torch.abs((target - output) / target))
48 optimizer = optim.Adam(model.parameters(), lr=0.0001)
49
50 n_epochs = 100 # number of epochs to run
51 batch_size = 10 # size of each batch
52 batch_start = torch.arange(0, len(X_train), batch_size)
53
54 # Hold the best model
55 best_mape = np.inf # init to infinity
56 best_weights = None
57
58 for epoch in range(n_epochs):
59     model.train()
60     for start in batch_start:
61         # take a batch
62         X_batch = X_train[start:start+batch_size]
63         y_batch = y_train[start:start+batch_size]
64         # forward pass
65         y_pred = model(X_batch)
66         loss = loss_fn(y_pred, y_batch)
67         # backward pass
68         optimizer.zero_grad()
69         loss.backward()
70         # update weights
71         optimizer.step()
72     # evaluate accuracy at end of each epoch
73     model.eval()
74     y_pred = model(X_test)
75     mape = float(loss_fn(y_pred, y_test))
76     if mape < best_mape:
77         best_mape = mape
78         best_weights = copy.deepcopy(model.state_dict())
79
80 # restore model and return best accuracy

```



```

81 model.load_state_dict(best_weights)
82 print("MAPE: %.2f" % best_mape)
83
84 model.eval()
85 with torch.no_grad():
86     # Test out inference with 5 samples
87     for i in range(5):
88         X_sample = X_test_raw[i:i+1]
89         X_sample = scaler.transform(X_sample)
90         X_sample = torch.tensor(X_sample, dtype=torch.float32)
91         y_pred = model(X_sample)
92         print(f"{X_test_raw[i]} -> {y_pred[0].numpy()} (expected {y_test[i].numpy()})")

```

Compare to the example in the other post, you can see that `loss_fn` now is defined as a custom function. Otherwise everything is just the same.

Further Readings

Below are the documentations from PyTorch that give you more details on how the various loss functions are implemented:

- `nn.L1Loss` from PyTorch
- `nn.MSELoss` from PyTorch
- `nn.CrossEntropyLoss` from PyTorch
- `nn.BCELoss` from PyTorch
- `nn.NLLLoss` from PyTorch

Summary

In this post, you have seen loss functions and the role that they play in a neural network. You have also seen some popular loss functions used in regression and classification models, as well as how to implement your own loss function for your PyTorch model. Specifically, you learned:

- What are loss functions, and why they are important in training
- Common loss functions for regression and classification problems
- How to use loss functions in your PyTorch model

Get Started on Deep Learning with PyTorch!

Learn how to build deep learning models

...using the newly released PyTorch 2.0 library

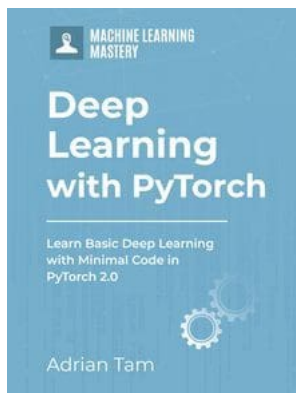
Discover how in my new Ebook:

Deep Learning with PyTorch

It provides **self-study tutorials** with **hundreds of working code** to turn you from a novice to expert. It equips you with *tensor operation, training, evaluation, hyperparameter optimization*, and much more...

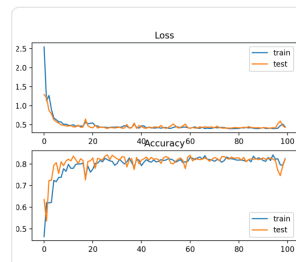
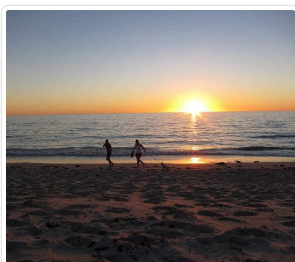
Kick-start your deep learning journey with hands-on exercises

SEE WHAT'S INSIDE



Share Share Post

More On This Topic



[Loss and Loss Functions for Training Deep Learning...](#)[Creating Custom Layers and Loss Functions in PyTorch](#)[How to Choose Loss Functions When Training](#)[How to Code the GAN Training Algorithm and Loss](#)[A Gentle Introduction to Generative Adversarial...](#)[A Gentle Introduction to XGBoost Loss Functions](#)

About Adrian Tam

Adrian Tam, PhD is a data scientist and software engineer.

[View all posts by Adrian Tam →](#)

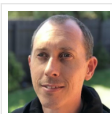
[< Using Activation Functions in Deep Learning Models](#)[Using Dropout Regularization in PyTorch Models >](#)

No comments yet.

Leave a Reply

 Name (required) Email (will not be published) (required)

SUBMIT COMMENT



Welcome!

I'm *Jason Brownlee* PhD
and I **help developers** get results with **machine learning**.
[Read more](#)

Never miss a tutorial:



Picked for you:



PyTorch Tutorial: How to Develop Deep Learning Models with Python



LSTM for Time Series Prediction in PyTorch



Deep Learning with PyTorch (9-Day Mini-Course)



Calculating Derivatives in PyTorch



Develop Your First Neural Network with PyTorch, Step by Step

Loving the Tutorials?

The **Deep Learning with PyTorch** EBook
is where you'll find the **Really Good** stuff.

>> SEE WHAT'S INSIDE



Machine Learning Mastery is part of Guiding Tech Media, a leading digital media publisher focused on helping people figure out technology. Visit our corporate [website](#) to learn more about our mission and team.



[PRIVACY](#) | [DISCLAIMER](#) | [TERMS](#) | [CONTACT](#) | [SITEMAP](#) | [ADVERTISE WITH US](#)

© 2026 Guiding Tech Media All Rights Reserved